

Table of contents

L'essentiel	2
Introduction et Motivation	2
Le problème fondamental	2
Intuition géométrique : le nuage de points	2
Maximisation de la variance	3
Formalisation mathématique	3
Décomposition complète	3
Minimisation de l'erreur	4
Équivalence variance-erreur	4
Interprétations	5
Réduction de Dimension	5
Structure de l'espace latent	5
Coordonnées latentes	5
Approximation de rang faible	6
Théorème d'Eckart et Young	6
Troncation pratique	6
Géométrie et qualité de reconstruction	7
Contribution relative à la variance	7
Ratio de projection	7
Normalisation et mise à l'échelle	7
Application pratique de la PCA	8
Algorithme PCA complet	8
Choix du nombre de composantes K	8
AutoEncodeurs linéaires	9
Structure d'un AutoEncodeur linéaire	9
Extensions de la PCA	10
PCA Probabiliste	10
Kernel PCA	10
Local PCA	10
Ce que la PCA vous dit (et ne vous dit pas)	10
Ce qu'elle vous dit	10
Ce qu'elle ne vous dit pas	11
Quand utiliser la PCA ?	11
Adapté pour	11
Non adapté pour	11

L'essentiel

Introduction et Motivation

La **PCA** (Analyse en Composantes Principales) est une méthode de **décomposition latente linéaire** qui permet d'identifier les **facteurs latents sous-jacents** structurant les données.

Le problème fondamental

Les représentations de base des données ne sont pas toujours optimales. La PCA cherche à trouver une nouvelle représentation qui :

- Révèle les **facteurs latents** (cachés) qui gouvernent le processus
- Utilise la **corrélation statistique** pour identifier ces facteurs
- Permet une **simplification** des données par décimation justifiée

Point clé : La PCA fait partie de la famille des **modèles latents linéaires**

Intuition géométrique : le nuage de points

Imaginez un nuage de points en 3D (comme des moucheron dans une pièce). Si vous deviez les écraser sur un mur :

- Quel mur choisiriez-vous pour que les moucheron soient **le plus étalés** ?
- Ce mur, c'est le **plan principal**
- La ligne dans ce mur où ils sont le plus alignés, c'est la **première composante**

Idée essentielle : La PCA trouve la **structure squelettique** des données, comme un radiologue qui voit les os à travers la peau.

Maximisation de la variance

Formalisation mathématique

Idée : On cherche une droite passant par le centre où les projections sont **le plus dispersées possible**.

Soit X l'ensemble de données, $\{u_i\}_{i \in [[D]]}$ une nouvelle **base orthonormale** de $\mathbb{R}^D$.

La **première composante principale** u_1 est choisie telle que la **variance des données projetées** sur u_1 soit maximale.

i Détails mathématiques : Problème d'optimisation

La moyenne empirique et la variance projetée sont :

$$x = \frac{1}{N} \sum_{i=1}^N x_i$$

$$v_{u_1} = \frac{1}{N} \sum_{i=1}^N (u_1^T x_i - u_1^T x)^2 = u_1^T \Sigma u_1$$

où $\Sigma = \frac{1}{N} \sum_{i=1}^N (x_i - x)(x_i - x)^T$ est la **matrice de covariance** des données.
Le problème devient :

$$u_1 = \arg \max_{u^T u = 1} u^T \Sigma u$$

En utilisant les **multiplicateurs de Lagrange** :

$$J(u) = u^T \Sigma u + \lambda(1 - u^T u)$$

Les conditions d'optimalité donnent :

$$\Sigma u_1 = \lambda_1 u_1 \quad \Rightarrow \quad v_{u_1} = \lambda_1$$

Conclusion : (u_1, λ_1) est une **paire propre** (eigenpair) de la matrice de covariance Σ

Décomposition complète

En continuant ce processus, on obtient l'ensemble des **composantes principales** comme les paires propres $\{(u_i, \lambda_i)\}_{i \in [[D]]}$ de Σ .

Décomposition matricielle :

$$\Lambda = U^T \Sigma U$$

où U contient les vecteurs propres en colonnes et Λ est diagonale avec les valeurs propres.

Minimisation de l'erreur

Idée : Représenter chaque point par sa projection sur une droite avec **l'erreur de reconstruction minimale** (distance perpendiculaire).

Équivalence variance-erreur

La variance v_{u_1} s'exprime comme une somme de carrés :

$$v_{u_1} = \frac{1}{N} \sum_{i=1}^N (u_1^T (x_i - x))^2$$

i Détails mathématiques : Théorème de Pythagore

Pour maximiser v_{u_1} , les termes $u_1^T (x_i - x)$ doivent être collectivement maximisés. Puisque $(x_i - x)$ est fixé, cela équivaut à **minimiser la distance à l'axe de projection** (erreur d'approximation).

Formulation alternative :

$$u_1 = \arg \min_{u^T u = 1} \sum_{i=1}^N \|(x_i - x) - [u^T (x_i - x)]u\|_2^2$$

Cette formulation donne également : $\Sigma u_1 = \lambda_1 u_1$

Équivalence fondamentale :

Maximiser la variance de projection Minimiser l'erreur d'approximation

Interprétations

- **Régression** : Une composante principale est une **régression quadratique** sur les données
 - **Physique** : Une composante principale est un **sous-espace d'inertie minimale** par rapport à X
-

Réduction de Dimension

Structure de l'espace latent

L'espace latent avec base $\{u_1, \dots, u_D\}$ possède les propriétés suivantes :

- **Ordre des valeurs propres** : Par construction : $\lambda_1 > \lambda_2 > \dots > \lambda_D$
- **Variance totale** : $\lambda = \sum_{d=1}^D \lambda_d = \text{Tr}(\Sigma)$
- **Décorrélacion** : $v_{u_d} = \lambda_d$ et $u_d^T u_{d'} = 0$ pour $d \neq d'$

Coordonnées latentes

La base $\{u_1, \dots, u_D\}$ induit des **coordonnées latentes** y_i pour les données :

$$y_i = U^T(x_i - x)$$

La transformation PCA est **linéaire** et composée de trois étapes :

- **Centrage** sur x
- **Rotation** utilisant la matrice U
- **Mise à l'échelle** (whitening) utilisant $\Lambda^{1/2}$

Limitation importante : La PCA suppose une distribution approximativement **gaussienne** et ne sera pas pertinente pour des données non-gaussiennes (par exemple, des données clusterisées).

Approximation de rang faible

Théorème d'Eckart et Young

Si $\Sigma = U\Lambda U^T$ et pour $K < D$, on définit :

$$\Sigma_K = \sum_{d=1}^K \lambda_d u_d u_d^T$$

Alors Σ_K est la **matrice de rang K la plus proche** de Σ au sens de la norme de Frobenius.

i Détails mathématiques : Erreur d'approximation

Le théorème stipule que :

$$\arg \min_{\text{rang}(S)=K} \|\Sigma - S\|_F^2 = \Sigma_K$$

où $\|\cdot\|_F$ est la **norme de Frobenius**.

L'erreur d'approximation est :

$$\|\Sigma - \Sigma_K\|_F^2 = \sum_{d=K+1}^D \lambda_d$$

Cette erreur correspond exactement à la variance perdue lors de la réduction.

Troncation pratique

Troncation de U et Λ :

$$\Sigma_K = U_K \Lambda_K U_K^T$$

Projection et reconstruction :

- **Projection** (encodage) : $\tilde{y}_i = U_K^T x_i \in \mathbb{R}^K$
- **Reconstruction** (décodage) : $\tilde{x}_i = U_K U_K^T x_i + x$

Important : On passe de D dimensions à K dimensions avec $K < D$

Géométrie et qualité de reconstruction

Contribution relative à la variance

La contribution de chaque dimension :

$$c_d = \frac{\lambda_d}{\sum_k \lambda_k}$$

Cette métrique dépend de la **distribution du spectre** des valeurs propres.

Ratio de projection

Le ratio de projection d'une donnée x_i sur un facteur latent :

$$\rho_d(x_i) = \frac{\langle u_d, x_i \rangle^2}{\|x_i\|^2} = \cos^2(\angle(u_d, x_i))$$

Plus $\rho_d(x_i)$ est proche de 1, plus x_i se situe sur l'axe u_d .

Normalisation et mise à l'échelle

La PCA est plus efficace si **toutes les échelles sont similaires**.

Solution : Normalisation

On crée la **matrice de mise à l'échelle** $S = \text{diag}[\sigma_1^2, \dots, \sigma_D^2]$ et on utilise la **métrique de Mahalanobis** :

$$d_S^2(x, y) = (x - y)^T S^{-1} (x - y)$$

Astuce pratique : Normaliser les données avant PCA améliore souvent les résultats.

Application pratique de la PCA

Algorithme PCA complet

Soit $X \in \Omega$ l'ensemble de données :

- Calculer la moyenne empirique $x = \frac{1}{N} \sum_{i=1}^N x_i$
- Centrer les données : $x_i \leftarrow (x_i - x)$ et former la matrice centrée X
- Calculer la matrice de covariance : $\Sigma = \frac{1}{N} X X^T$
- Décomposer en valeurs propres : $\Sigma = U \Lambda U^T$

Jusqu'ici : transformation **exacte**

- Sélectionner le nombre de composantes K
- Définir U_K , Λ_K et calculer $\{\tilde{y}_i\}_{i \in [[N]]}$ et/ou $\{\tilde{x}_i\}_{i \in [[N]]}$

Choix du nombre de composantes K

Trois stratégies principales :

Stratégie 1 : Dimension cible

$$K = d$$

où d est la dimension souhaitée.

Stratégie 2 : Seuil de variance

Exiger que l'espace latent conserve une proportion τ de la variance totale :

$$\frac{\sum_{d=1}^K \lambda_d}{\sum_{d=1}^D \lambda_d} \geq \tau$$

Exemple : $\tau = 0.95$ signifie conserver 95% de la variance.

Stratégie 3 : Erreur de reconstruction

Entraîner K tel que $L(X) \leq \epsilon$, où par exemple :

$$L(X) = \sum_i \|x_i - \tilde{x}_i\|^2$$

Conseil pratique : Visualiser le spectre des valeurs propres (scree plot) aide à identifier un “coude” naturel.

AutoEncodeurs linéaires

Structure d'un AutoEncodeur linéaire

Un AutoEncodeur linéaire possède :

- W_{enc} et W_{dec} : matrices de poids (encoder et decoder)
- $z(x_i) = W_{\text{enc}}x_i$: encodage
- $\tilde{x}_i = W_{\text{dec}}z(x_i)$: décodage

Problème d'optimisation :

$$\theta^* = \arg \min_{\text{rang}(W)=K} \|X - WZ\|_F^2$$

i Détails mathématiques : Relation PCA et AutoEncodeur

En utilisant le théorème d'Eckart et Young avec la décomposition SVD $X = U\Psi V^T$, la solution est :

$$W_{\text{dec}}Z = U_K\Psi_K V_K^T$$

En posant $W_{\text{dec}} = U_K\Psi_K$, alors :

$$W_{\text{enc}} = \Psi_K^{-1}U_K^T$$

Pour des données **centrées** :

- **PCA** : $\Sigma = \frac{1}{N}XX^T = U\Lambda U^T$ donc $\tilde{x}_i = U_K U_K^T x_i$
- **AE** : $X = U\Psi V^T$ donc $\Sigma = \frac{1}{N}U\Psi^2 U^T$ et $\tilde{x}_i = U_K U_K^T x_i$

Relation entre valeurs propres :

$$\Lambda = \frac{1}{N}\Psi^2$$

Conclusion fondamentale : Un AutoEncodeur linéaire effectue une PCA si les données sont **centrées et normalisées**.

Note : W_{enc} n'est **pas l'inverse** de W_{dec} si $K < D$.

Extensions de la PCA

PCA Probabiliste

Reformulation avec distribution latente explicite :

- Distribution latente : $\mathbb{P}(z) = \mathcal{N}(0, I_{dK})$
- Modèle conditionnel : $\mathbb{P}(x|z) = \mathcal{N}(Wz + \mu, \sigma^2 I_{dD})$

Avantages :

- Accommodation du **bruit de mesure** (variance σ^2)
- Mode **génératif** : possibilité de générer de nouvelles données
- Estimation par **Maximum de Vraisemblance**
- Algorithme **EM** pour économiser les calculs

Kernel PCA

Utilisation d'un **mapping non-linéaire** $\phi(x_i)$ via une fonction noyau :

$$k(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

Permet d'effectuer la PCA dans un **espace plus favorable** (possiblement de dimension infinie) sans calculer explicitement ϕ .

Local PCA

Effectue la PCA sur des **voisinages locaux** des données pour considérer uniquement la **dimensionnalité intrinsèque locale**.

Utile pour des données avec structure locale complexe (variétés non-linéaires).

Ce que la PCA vous dit (et ne vous dit pas)

Ce qu'elle vous dit

- La **structure linéaire** des données
- Les **directions principales** de variation
- La **dimension intrinsèque** (combien de dimensions vraiment utiles)
- Quelles **variables** contribuent à chaque axe

Ce qu'elle ne vous dit pas

- **Pas un clustering** : Elle ne trouve pas des groupes
 - **Linéaire uniquement** : Elle rate les relations courbes
 - **Sensible aux outliers** : Un point extrême peut tout faire basculer
-

Quand utiliser la PCA ?

Adapté pour

- Données suivant approximativement une distribution gaussienne
- Réduction de dimension avec préservation de variance
- Visualisation de données haute dimension
- Débruitage de données corrompues par bruit gaussien
- Compression de données

Non adapté pour

- Données fortement clusterisées (non-gaussiennes)
 - Données où les relations sont fortement non-linéaires
 - Cas où la variance n'est pas un bon critère d'information
-

Fiche de révision condensée

En 30 secondes :

- PCA = changement de base intelligent
- Nouveaux axes = directions de variance maximale
- Valeurs propres = importance de chaque axe
- Toujours centrer d'abord !
- Choisir K avec le scree plot
- Utile pour : visualisation, réduction dimension, débruitage
- Pas utile pour : clustering, relations non-linéaires

! Concepts mathématiques à maîtriser

Pour bien comprendre la PCA :

- **Transformation linéaire et matrice orthogonale**
- **Coordonnées et rang** matriciel
- **Moyenne et variance**
- **Projection** orthogonale
- **Décomposition en valeurs propres** (eigendecomposition)
- **Trace** d'une matrice
- **Norme de Frobenius**
- **Multiplicateurs de Lagrange**